

Alia 4 LED Controller - Tutorial

This guide gets you from zero to a running board, then walks through writing your own LED patterns with several ready-to-use examples.

Part 1: Setup

1. Get the code

- Visit github.com/johncohn/alia_blinky → green **Code** button → **Download ZIP**, then unzip it
- Or: `git clone https://github.com/johncohn/alia_blinky.git`

2. Install Arduino IDE 2.x from arduino.cc

3. Install the board core — Tools → Board → Boards Manager, then search and install:

- **RP2040 boards** (Seeed XIAO RP2040, Adafruit QT Py RP2040): search `rp2040`, install "Raspberry Pi Pico/RP2040" (v5.4.3+)
- **ESP32-S3 boards** (Seeed XIAO ESP32-S3, Adafruit QT Py ESP32-S3): search `esp32`, install "esp32 by Espressif Systems" (v3.3.3+)

4. Install the library — Tools → Manage Libraries → search Adafruit NeoPixel → Install (v1.15.2+)

5. Open and upload

- File → Open → `Alia_blinky_esp32.ino` from the folder you downloaded in step 1
- Tools → Board → select your specific board
- Plug in the board, select its port under Tools → Port, then click **Upload** (→)

6. Verify it's running — Tools → Serial Monitor, set baud rate to **115200**. You should see the board type detected and pattern names printed as it cycles.

Troubleshooting: if upload fails on an RP2040 board, double-tap its RESET button to force bootloader mode (a drive named RPI-RP2 should appear), then upload again.

Part 2: Writing Your Own Pattern

Every custom pattern needs 4 edits to `Alia_blinky_esp32.ino`:

#	What	Where
1	Write your pattern function	CUSTOM PATTERNS SECTION (~line 1075)
2	Add one to NUM_PATTERNS	~line 91
3	Add a case for it in the switch	loop() (~line 1268)
4	Add its name to subModeNames[]	~line 1254

Rules of thumb: case numbers must be sequential starting at 0 with no gaps, and NUM_PATTERNS must equal the total count. The 4 built-in patterns already occupy cases 0-3, so your first custom pattern is **case 4**.

Worked Example: Red, White & Blue

Let's add a pattern together, step by step.

Step 1 — write the function. Paste this into the CUSTOM PATTERNS SECTION (~line 1075):

```
// RED WHITE BLUE - patriotic stripes marching down the string
void redWhiteBluePattern() {
    static int offset = 0;
    static unsigned long lastUpdate = 0;

    if (millis() - lastUpdate > 100) {
        for (int i = 0; i < LED_COUNT; i++) {
            int stripe = (i + offset) % 3;
            if (stripe == 0) strip.setPixelColor(i, strip.Color(bright, 0, 0));
            else if (stripe == 1) strip.setPixelColor(i, strip.Color(0, bright, 0));
            else strip.setPixelColor(i, strip.Color(0, 0, bright));
        }
        strip.show();
        lights();
        offset++;
        lastUpdate = millis();
    }
}
```

Step 2 — bump the pattern count. At ~line 91, change:

```
#define NUM_PATTERNS 4
```

to:

```
#define NUM_PATTERNS 5 // added RED WHITE BLUE
```

Step 3 — add a case. In the switch in loop() (~line 1268), add after case 3:

```
case 4:
    redWhiteBluePattern();
    break;
```

Step 4 — name it. In subModeNames[] (~line 1254), add:

```
const char* subModeNames[] = {
    "FLIGHT", "SLOW RAINBOW", "FAST WHITE", "RAINBOW PROPS",
    "RED WHITE BLUE" // Pattern 4 - new!
};
```

Step 5 — test. Upload, open Serial Monitor (115200 baud), and wait for the auto-cycle to print RED WHITE BLUE — that's your pattern running.

More Examples

Each of these drops in the same way — paste the function, add its case, add its name, bump NUM_PATTERNS. Add as many or as few as you like; just keep the case numbers sequential.

Dot Chase — one white LED travels down the entire string (all 4 props, then the tail) and loops:

```
void dotChasePattern() {
    static int currentLED = 0;
    static unsigned long lastUpdate = 0;

    if (millis() - lastUpdate > 30) {
        strip.clear();
        strip.setPixelColor(currentLED, strip.Color(brightness, br
        strip.show();
        lights();

        currentLED++;
        if (currentLED >= LED_COUNT) currentLED = 0;
        lastUpdate = millis();
    }
}
```

case 5: dotChasePattern(); break; — name: "DOT CHASE"

Rainbow Props (Solid) — each prop is a solid color; all 4 slowly cycle through the rainbow, offset so no two props ever match:

```
void rainbowPropsSolidPattern() {
    static uint16_t baseHue = 0;
    static unsigned long lastUpdate = 0;

    if (millis() - lastUpdate > 20) {
        uint16_t hueStep = 65536 / 4; // split the color wheel 4

        for (int prop = 0; prop < 4; prop++) {
            uint32_t color = strip.gamma32(strip.ColorHSV(baseHue +
                for (int led = 0; led < 9; led++) {
                    strip.setPixelColor((prop * 9) + led, color);
                }
            }
            strip.show();
            lights();
            baseHue += 100;
            lastUpdate = millis();
        }
    }
}
```

case 6: rainbowPropsSolidPattern(); break; — name: "RAINBOW PROPS SOLID"

Sparkle — random LEDs twinkle white against a dark background:

```
void sparklePattern() {
    static unsigned long lastUpdate = 0;

    if (millis() - lastUpdate > 50) {
        strip.clear();
        for (int i = 0; i < 6; i++) { // 6 sparkles at a time
            strip.setPixelColor(random(LED_COUNT), strip.Color(brigh
        }
        strip.show();
        lights();
        lastUpdate = millis();
    }
}
```

case 7: sparklePattern(); break; — name: "SPARKLE"

Fire — random orange/red flicker across all LEDs:

```

void firePattern() {
    static unsigned long lastUpdate = 0;

    if (millis() - lastUpdate > 40) {
        for (int i = 0; i < LED_COUNT; i++) {
            int flicker = random(brightness / 2, brightness);
            strip.setPixelColor(i, strip.Color(flicker, flicker / 4,
            }
            strip.show();
            lights();
            lastUpdate = millis();
        }
    }
}

```

case 8: firePattern(); break; — name: "FIRE"

Part 3: Removing Patterns

Don't like one of the built-in patterns? Same 4 spots, in reverse:

1. Delete (or comment out) its case in the switch
2. **Renumber the remaining cases** so they stay sequential from 0
3. Remove its name from subModeNames[]
4. Decrease NUM_PATTERNS by 1

To keep only **FLIGHT** running on a loop: set NUM_PATTERNS 1, keep just case 0, and set subModeNames[] to { "FLIGHT" }.

Part 4: Tips and Common Mistakes

- **Use millis(), not delay(),** for timing (see the examples above) — long delay() calls freeze the whole board and make animations choppy.
- **Always call strip.show()** after setting pixel colors, or nothing will update.
- **Call lights()** (or lights(false)) so the nose/wingtip nav lights stay in sync with your pattern.
- **NUM_PATTERNS mismatch** is the most common bug — if your new pattern never appears, check it first.
- **Case numbers must be sequential** (0, 1, 2, 3...) with no gaps or duplicates.

Study the FLIGHT Pattern

flightPattern() (~line 723) is the most advanced pattern in the file — a multi-phase state machine with non-linear (exponential) motion curves and completion-

based timing instead of a fixed duration. Worth reading once you're comfortable with the basics above.

Share Your Patterns

Fork the repo, add your patterns, and submit a pull request — help others learn!

See Also

README.md	General info about the board, hardware, and features
SOLDER.md	Photo guide for soldering and gluing the PCB assembly